

Unidad Didáctica

**Implementación de Base de Datos,
Unidad IV**

Contenido

Introducción	4
Desarrollo de Contenidos	5
Lenguaje de Definición de Objetos	5
Create.....	6
Alter	7
Drop	8
Identificadores.....	10
Tipos de Datos.....	11
Creación y Eliminación.....	14
Tipos de datos definidos por el usuario	15
Tabla	17
Columna	19
Uso de propiedad: identity, función newid	20
Tipos de integridad	22
Integridad de dominio.....	22
Integridad de entidad	23
Integridad referencial.....	24
Restricciones	26
Default.....	26
Check	28

Primary Key.....	29
Unique.....	30
Foreign Key.....	31
Integridad referencial en cascada	33
Bibliografía y Webgrafía	34
GlosarioBibliografía y Webgrafía	34
Glosario	35

Introducción



¡Bienvenido/a! En el mundo de las bases de datos, el Lenguaje de Definición de Objetos es una parte fundamental. Este lenguaje permite crear, modificar y eliminar objetos en una base de datos, como tablas, columnas y restricciones. Entre las acciones que se pueden realizar están Create, Alter y Drop, que permiten crear objetos nuevos, modificar objetos existentes y eliminar objetos, respectivamente.

Para trabajar con Lenguaje de Definición de Objetos es importante conocer los identificadores y los tipos de datos. Los identificadores son nombres únicos que se utilizan para identificar objetos en la base de datos. Los tipos de datos definen el tipo de información que se puede almacenar en una columna de una tabla, como números, texto o fechas. Además, es posible crear y eliminar tipos de datos definidos por el usuario, para adaptar la base de datos a necesidades específicas.

En cuanto a las restricciones, son una parte importante de la integridad de los datos. Las restricciones se utilizan para definir reglas que los datos deben cumplir, como valores permitidos, valores predeterminados, claves primarias y claves foráneas. Además, la integridad referencial asegura que los datos en diferentes tablas estén relacionados de forma correcta. En resumen, el Lenguaje de Definición de Objetos es esencial para trabajar con bases de datos y las restricciones son una parte importante de la integridad de los datos.

Lenguaje de Definición de Objetos

El Lenguaje de Definición de Objetos (ODL, por sus siglas en inglés) es un lenguaje de programación orientado a objetos utilizado para definir objetos y relaciones en una base de datos orientada a objetos. ODL se utiliza para crear esquemas de base de datos y definir la estructura de los objetos almacenados en una base de datos.

El objetivo principal de ODL es proporcionar una forma estandarizada y estructurada de definir objetos en una base de datos orientada a objetos. Los objetos en una base de datos orientada a objetos se definen como entidades independientes que tienen propiedades y métodos únicos. Estos objetos se utilizan para representar datos del mundo real y pueden ser accedidos y manipulados mediante programas de aplicación. ODL se basa en la sintaxis del lenguaje de programación C++. El lenguaje utiliza un conjunto de declaraciones para definir la estructura de los objetos, incluyendo sus propiedades y métodos. Los objetos en ODL se pueden heredar y pueden ser utilizados para construir relaciones complejas entre objetos en una base de datos.

Una de las principales ventajas de ODL es su capacidad para representar objetos complejos en una base de datos de forma clara y estructurada. Los objetos pueden ser fácilmente definidos y manipulados, lo que permite a los desarrolladores de aplicaciones trabajar de forma eficiente con datos orientados a objetos. Un ejemplo de ODL sería la definición de un objeto "Cliente" en una base de datos de una empresa de comercio electrónico. El objeto "Cliente" tendría propiedades como nombre, dirección y número de teléfono, así como métodos como "realizar un pedido" y "ver historial de pedidos". Estas propiedades y métodos se definirían en ODL utilizando una sintaxis estructurada y fácil de leer.

Create

La creación de una instancia de SQL Server es un proceso crítico en la implementación de una base de datos en una organización. SQL Server es un sistema de gestión de bases de datos relacionales desarrollado por Microsoft y es ampliamente utilizado en el mundo empresarial para administrar grandes cantidades de datos y proporcionar soluciones de almacenamiento de datos escalables y seguras. En este contexto, el proceso de creación de una instancia de SQL Server implica la instalación y configuración de la plataforma en un servidor determinado.

El primer paso en la creación de una instancia de SQL Server es descargar la versión más reciente del software desde el sitio web de Microsoft. Después de descargar el software, se debe iniciar la instalación, lo que implica la selección de una instancia de SQL Server apropiada, la elección de una ubicación para la base de datos, la selección de una opción de autenticación y la configuración de otros parámetros de configuración.

Una vez completada la instalación, se puede crear una base de datos en SQL Server. Esto se hace utilizando el lenguaje SQL (Structured Query Language), que es el lenguaje estándar utilizado para interactuar con bases de datos relacionales. Para crear una base de datos, se deben seguir los siguientes pasos:

1. Abrir SQL Server Management Studio, la interfaz gráfica de usuario para administrar SQL Server.
2. Conectar a la instancia de SQL Server deseada.
3. Hacer clic en "New Query" para abrir una nueva ventana de consulta.
4. Escribir una consulta SQL que incluya la instrucción "CREATE DATABASE" seguida del nombre de la base de datos que se desea crear.
5. Ejecutar la consulta SQL haciendo clic en el botón "Ejecutar".

Un ejemplo de una consulta SQL para crear una base de datos en SQL Server podría ser:

```
CREATE DATABASE Sales
```

Esta consulta crearía una base de datos llamada "Sales" en la instancia de SQL Server a la que se ha conectado el usuario.

Alter

La alteración de una base de datos es un proceso crítico en la gestión de una base de datos relacionales en SQL Server. SQL Server es un sistema de gestión de bases de datos relacionales desarrollado por Microsoft y es ampliamente utilizado en el mundo empresarial para administrar grandes cantidades de datos y proporcionar soluciones de almacenamiento de datos escalables y seguras. En este contexto, la alteración de una base de datos en SQL Server implica realizar cambios en la estructura o en los datos de una base de datos existente.

La alteración de una base de datos en SQL Server se puede realizar mediante el uso de la instrucción "ALTER". La instrucción "ALTER" se utiliza para modificar la estructura de una tabla existente, agregar o eliminar columnas, cambiar los tipos de datos, agregar o eliminar índices y realizar otros cambios relacionados con la estructura de la tabla. La sintaxis general de la instrucción "ALTER" es la siguiente:

```
ALTER TABLE table_name  
ADD column_name datatype constraint;
```

Donde "table_name" es el nombre de la tabla a la que se aplicará la instrucción "ALTER", "column_name" es el nombre de la columna que se agregará y "datatype"

es el tipo de datos de la columna. El parámetro "constraint" es opcional y se utiliza para agregar restricciones a la columna.

Un ejemplo de una instrucción "ALTER" en SQL Server podría ser:

```
ALTER TABLE Customers  
ADD Email varchar(255) NOT NULL;
```

Esta instrucción agrega una nueva columna llamada "Email" a la tabla "Customers" con un tipo de datos "varchar" y una longitud máxima de 255 caracteres. La cláusula "NOT NULL" indica que la columna no puede contener valores nulos.

Además de la instrucción "ALTER TABLE", también existe la instrucción "ALTER DATABASE" que se utiliza para realizar cambios en la base de datos en su conjunto. Esta instrucción se utiliza para modificar la configuración de la base de datos, agregar o eliminar archivos de datos o de registro y realizar otros cambios relacionados con la estructura de la base de datos.

Drop

El proceso de eliminación de una base de datos en SQL Server se conoce como "DROP". SQL Server es un sistema de gestión de bases de datos relacionales desarrollado por Microsoft y es ampliamente utilizado en el mundo empresarial para administrar grandes cantidades de datos y proporcionar soluciones de almacenamiento de datos escalables y seguras. En este contexto, la eliminación de una base de datos en SQL Server implica la eliminación de la base de datos completa, incluyendo todas las tablas, columnas y datos asociados.

La eliminación de una base de datos en SQL Server se puede realizar mediante el uso de la instrucción "DROP DATABASE". La instrucción "DROP DATABASE" se utiliza para eliminar una base de datos completa y todos sus objetos asociados, como tablas, vistas, procedimientos almacenados, etc. La sintaxis general de la instrucción "DROP DATABASE" es la siguiente:

```
DROP DATABASE database_name;
```

Donde "database_name" es el nombre de la base de datos que se desea eliminar.

Es importante tener en cuenta que la eliminación de una base de datos en SQL Server es un proceso irreversible y que todos los datos almacenados en la base de datos se perderán. Por lo tanto, es recomendable hacer una copia de seguridad de la base de datos antes de proceder con la eliminación.

Un ejemplo de una instrucción "DROP DATABASE" en SQL Server podría ser:

```
DROP DATABASE Sales;
```

Esta instrucción eliminaría completamente la base de datos "Sales" y todos sus objetos asociados. Es importante tener en cuenta que la eliminación de una base de datos es una operación crítica y debe realizarse con precaución. Se recomienda encarecidamente que el usuario tenga los permisos necesarios para realizar esta tarea y que realice una copia de seguridad de la base de datos antes de proceder con la eliminación. Además, es importante asegurarse de que no se dependa de la base de datos que se va a eliminar y que no haya procesos en ejecución que estén utilizando la base de datos en ese momento.

Identificadores

En SQL Server, los identificadores son utilizados para nombrar objetos de bases de datos, como tablas, columnas, vistas y procedimientos almacenados. Los identificadores en SQL Server deben seguir ciertas reglas y convenciones para asegurar su correcto funcionamiento y evitar errores al momento de trabajar con ellos.

Los identificadores en SQL Server pueden estar compuestos por letras, números y caracteres especiales, aunque se recomienda evitar el uso de caracteres especiales para facilitar la lectura y el mantenimiento del código. Los identificadores también deben comenzar con una letra y no pueden ser una palabra reservada de SQL Server, como "SELECT", "INSERT", "UPDATE", entre otras.

Para utilizar un identificador que contenga caracteres especiales o que comience con un número, se debe colocar entre corchetes []. Por ejemplo:

```
SELECT [my column], [2ndColumn], [my_column3]  
FROM my_table;
```

En este ejemplo, se utilizan identificadores que contienen espacios y comienzan con un número. Los corchetes permiten que el identificador sea utilizado sin causar errores de sintaxis.

Es importante tener en cuenta que los identificadores en SQL Server son sensibles a mayúsculas y minúsculas. Por lo tanto, un identificador escrito en mayúsculas no se considera igual a otro escrito en minúsculas. Por ejemplo:

```
SELECT [MyColumn] FROM my_table;
```

Esta consulta no es igual a la siguiente:

```
SELECT [mycolumn] FROM my_table;
```

Aunque ambos identificadores se refieren a la misma columna, la segunda consulta no es correcta debido a que se utiliza una letra minúscula en lugar de una mayúscula.

En resumen, los identificadores en SQL Server son utilizados para nombrar objetos de bases de datos y deben seguir ciertas reglas y convenciones para asegurar su correcto funcionamiento. Los identificadores pueden contener letras, números y caracteres especiales, y se recomienda evitar el uso de caracteres especiales para facilitar la lectura y el mantenimiento del código. Los identificadores también son sensibles a mayúsculas y minúsculas, por lo que se debe prestar atención a la forma en que se escriben para evitar errores de sintaxis.

Tipos de Datos

Esta tabla presenta los tipos de datos más comunes en SQL Server, sus descripciones, ejemplos y casos de uso recomendados. Sin embargo, existen otros tipos de datos en SQL Server que pueden ser útiles en situaciones específicas, por lo que es importante investigar y comprender bien las opciones disponibles antes de tomar una decisión.

<i>Tipo de dato</i>	Descripción	Ejemplo	Cuando usarlo
<i>int</i>	Entero de 4 bytes con signo	123456	Para almacenar valores enteros numéricos
<i>bigint</i>	Entero de 8 bytes con signo	9223372036854775807	Para almacenar valores enteros grandes numéricos
<i>smallint</i>	Entero de 2 bytes con signo	32767	Para almacenar valores enteros numéricos pequeños
<i>tinyint</i>	Entero de 1 byte sin signo	255	Para almacenar valores enteros numéricos pequeños y sin signo
<i>decimal(p,s)</i>	Número decimal con precisión p y escala s	123.45	Para almacenar valores numéricos con una precisión y escala específicas
<i>float(n)</i>	Número de punto flotante con precisión de n bits	123.456789	Para almacenar valores numéricos con una precisión variable
<i>datetime</i>	Fecha y hora	2023-04-09 15:30:00	Para almacenar fechas y horas
<i>date</i>	Fecha	2023-04-09	Para almacenar fechas sin hora
<i>time</i>	Hora	15:30:00	Para almacenar horas sin fecha

<i>char(n)</i>	Cadena de caracteres fija con longitud n	'hello'	Para almacenar cadenas de caracteres con una longitud fija
<i>varchar(n)</i>	Cadena de caracteres variable con longitud máxima n	'hello world'	Para almacenar cadenas de caracteres con una longitud variable
<i>nchar(n)</i>	Cadena de caracteres Unicode fija con longitud n	N'hola'	Para almacenar cadenas de caracteres Unicode con una longitud fija
<i>nvarchar(n)</i>	Cadena de caracteres Unicode variable con longitud máxima n	N'hola mundo'	Para almacenar cadenas de caracteres Unicode con una longitud variable
<i>binary(n)</i>	Datos binarios fijos con longitud n	0x1234	Para almacenar datos binarios con una longitud fija
<i>varbinary(n)</i>	Datos binarios variables con longitud máxima n	0x12345678	Para almacenar datos binarios con una longitud variable
<i>bit</i>	Valor booleano	1 o 0	Para almacenar valores booleanos

Creación y Eliminación

En SQL Server, la creación y eliminación de objetos es una tarea esencial para la administración de una base de datos. La creación implica la definición de una estructura de objeto en la base de datos, mientras que la eliminación implica la eliminación completa de un objeto y su estructura de la base de datos.

Para crear un objeto en SQL Server, se utiliza la instrucción CREATE seguida del tipo de objeto que se desea crear, como tabla, vista, función, procedimiento almacenado, entre otros. Luego, se define el nombre del objeto y sus atributos o propiedades. A continuación, se proporciona un ejemplo de creación de una tabla en SQL Server:

```
CREATE TABLE Ejemplo (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL,  
    edad INT,  
    email VARCHAR(100) UNIQUE  
);
```

En este ejemplo, se crea una tabla llamada "Ejemplo" con cuatro columnas: "id" de tipo entero, "nombre" de tipo cadena de caracteres con una longitud máxima de 50, "edad" de tipo entero y "email" de tipo cadena de caracteres con una longitud máxima de 100 y una restricción UNIQUE que garantiza que no se repitan valores en esa columna.

Para eliminar un objeto en SQL Server, se utiliza la instrucción DROP seguida del tipo de objeto que se desea eliminar, seguido del nombre del objeto. A continuación,

se proporciona un ejemplo de eliminación de la tabla "Ejemplo" creada anteriormente:

`DROP TABLE Ejemplo;`

Es importante tener en cuenta que la eliminación de un objeto es una acción permanente y que cualquier dato contenido en el objeto también se eliminará. Por lo tanto, es recomendable tener precaución al eliminar objetos de la base de datos.

En resumen, la creación y eliminación de objetos en SQL Server son acciones esenciales para administrar una base de datos. La instrucción CREATE se utiliza para definir una estructura de objeto en la base de datos, mientras que la instrucción DROP se utiliza para eliminar un objeto completo y su estructura. Es importante tener precaución al eliminar objetos de la base de datos, ya que la eliminación es una acción permanente.

Tipos de datos definidos por el usuario

SQL Server permite definir tipos de datos personalizados o definidos por el usuario para adaptarse a necesidades específicas que no son cubiertas por los tipos de datos predefinidos en el sistema. Los tipos de datos definidos por el usuario son utilizados en la definición de tablas, columnas y parámetros de procedimientos almacenados. La creación de un tipo de dato definido por el usuario en SQL Server se hace a través de la instrucción CREATE TYPE, seguida del nombre del tipo de dato y su definición. La definición puede ser una de tres opciones:

1. Basada en otro tipo de dato predefinido en SQL Server, como entero, cadena de caracteres, fecha, entre otros.
2. Basada en otro tipo de dato definido por el usuario previamente creado.

3. Definida a través de una tabla temporal.

A continuación, se proporciona un ejemplo de creación de un tipo de dato definido por el usuario en SQL Server basado en un tipo de dato predefinido:

```
CREATE TYPE MiTipoDeDato AS INT;
```

En este ejemplo, se crea un tipo de dato llamado "MiTipoDeDato" que es una copia del tipo de dato predefinido "INT". También es posible definir un tipo de dato definido por el usuario basado en otro tipo de dato definido por el usuario, como se muestra en el siguiente ejemplo:

```
CREATE TYPE MiOtroTipoDeDato FROM MiTipoDeDato;
```

En este ejemplo, se crea un tipo de dato llamado "MiOtroTipoDeDato" basado en el tipo de dato definido por el usuario previamente creado "MiTipoDeDato".

Además, es posible definir un tipo de dato definido por el usuario basado en una tabla temporal, como se muestra en el siguiente ejemplo:

```
CREATE TYPE MiTablaTemporalType AS TABLE (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL  
);
```

En este ejemplo, se crea un tipo de dato llamado "MiTablaTemporalType" basado en la estructura de una tabla temporal con dos columnas, "id" y "nombre".

En conclusión, los tipos de datos definidos por el usuario en SQL Server son una forma eficiente de adaptar la base de datos a necesidades específicas que no son cubiertas por los tipos de datos predefinidos. La creación de un tipo de dato definido por el usuario se hace a través de la instrucción CREATE TYPE, y la definición puede ser basada en otro tipo de dato predefinido en SQL Server, otro tipo de dato definido por el usuario previamente creado o una tabla temporal.

Tabla

En SQL Server, una tabla es un objeto que representa una colección de datos organizados en filas y columnas. Cada fila representa una instancia de los datos y cada columna representa un atributo o campo específico. Las tablas son utilizadas para almacenar y manipular datos en una base de datos relacional.

La creación de una tabla en SQL Server se realiza mediante la instrucción CREATE TABLE, seguida del nombre de la tabla y su definición. La definición de la tabla incluye el nombre de las columnas, el tipo de datos de cada columna y las restricciones que se aplican a las columnas, como las restricciones de integridad referencial y las restricciones de verificación. A continuación, se muestra un ejemplo de creación de una tabla simple en SQL Server:

```
CREATE TABLE Empleados (  
    id INT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL,  
    edad INT,  
    departamento VARCHAR(50)  
);
```

En este ejemplo, se crea una tabla llamada "Empleados" con cuatro columnas: "id", "nombre", "edad" y "departamento". La columna "id" se define como clave primaria y cada una de las columnas tiene un tipo de datos asociado. Las tablas en SQL Server también pueden tener restricciones adicionales, como restricciones de clave foránea, restricciones de verificación, restricciones de unicidad, entre otras. A continuación, se muestra un ejemplo de creación de una tabla con una restricción de clave foránea:

```
CREATE TABLE Ventas (  
    id INT PRIMARY KEY,  
    fechaVenta DATE,  
    idProducto INT,  
    cantidad INT,  
    FOREIGN KEY (idProducto) REFERENCES Productos(id)  
);
```

En este ejemplo, se crea una tabla llamada "Ventas" con cuatro columnas: "id", "fechaVenta", "idProducto" y "cantidad". La columna "idProducto" se define como clave foránea haciendo referencia a la tabla "Productos" en la columna "id".

En conclusión, las tablas en SQL Server son fundamentales en la creación y manipulación de datos en una base de datos relacional. La creación de una tabla se realiza mediante la instrucción CREATE TABLE, seguida de la definición de la tabla que incluye el nombre de las columnas, el tipo de datos de cada columna y las restricciones aplicables. Las tablas pueden incluir restricciones adicionales, como restricciones de clave foránea, restricciones de verificación y restricciones de unicidad para asegurar la integridad de los datos almacenados.

Columna

En SQL Server, una columna es un elemento estructural de una tabla que representa una categoría específica de datos. Cada columna tiene un nombre y un tipo de datos asociado que define el tipo de valor que puede contener. Además, cada columna puede estar sujeta a restricciones que regulan el tipo y la cantidad de datos que puede contener.

La creación de una columna se realiza en el contexto de la creación de una tabla. Para crear una columna, se utiliza la instrucción `ALTER TABLE`, seguida del nombre de la tabla y la definición de la nueva columna. La definición de una columna incluye el nombre de la columna, el tipo de datos asociado y cualquier restricción aplicable.

A continuación, se muestra un ejemplo de cómo agregar una nueva columna a una tabla en SQL Server:

```
ALTER TABLE Empleados  
ADD salario DECIMAL(10,2) NOT NULL;
```

En este ejemplo, se agrega una nueva columna llamada "salario" a la tabla "Empleados". La columna tiene un tipo de datos decimal con una precisión de 10 dígitos y una escala de 2 dígitos, y está sujeta a una restricción `NOT NULL`, lo que significa que la columna no puede contener valores nulos.

Las columnas también pueden estar sujetas a restricciones adicionales, como restricciones de clave foránea, restricciones de unicidad y restricciones de verificación. Estas restricciones pueden garantizar la integridad de los datos y evitar la entrada de valores incorrectos en la base de datos.

En conclusión, las columnas en SQL Server son elementos estructurales de una tabla que representan una categoría específica de datos. La creación de una columna se realiza en el contexto de la creación de una tabla mediante la instrucción ALTER TABLE y la definición de la columna que incluye el nombre de la columna, el tipo de datos asociado y cualquier restricción aplicable. Las columnas pueden estar sujetas a restricciones adicionales que garantizan la integridad de los datos en la base de datos.

Uso de propiedad: identity, función newid

En SQL Server, la propiedad "identity" y la función "newid" son herramientas utilizadas para generar valores únicos en las tablas de la base de datos. La propiedad "identity" es una propiedad de columna que se utiliza para generar automáticamente valores únicos y secuenciales para una columna determinada en una tabla. Para utilizar la propiedad "identity", es necesario definir la columna con un tipo de datos numérico y agregar la propiedad "identity" con la instrucción IDENTITY. La propiedad "identity" genera valores únicos y secuenciales para cada registro que se inserta en la tabla. A continuación, se muestra un ejemplo de cómo crear una tabla con la propiedad "identity" en SQL Server:

sql

Copy code

```
CREATE TABLE Ejemplo
```

```
(
```

```
    Id INT IDENTITY(1,1) PRIMARY KEY,
```

```
    Nombre VARCHAR(50) NOT NULL
```

```
);
```

En este ejemplo, se crea una tabla llamada "Ejemplo" con dos columnas: "Id" y "Nombre". La columna "Id" está definida como una propiedad "identity" con valores iniciales y de incremento de 1. La columna "Id" también se define como la clave primaria de la tabla.

La función "newid" es otra herramienta utilizada para generar valores únicos en SQL Server. La función "newid" devuelve un valor único de tipo uniqueidentifier, que se puede utilizar como valor predeterminado para una columna. Esta función se puede utilizar para generar identificadores únicos en una tabla sin la necesidad de definir una columna "identity".

A continuación, se muestra un ejemplo de cómo crear una tabla con la función "newid" en SQL Server:

```
CREATE TABLE Ejemplo  
(  
    Id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),  
    Nombre VARCHAR(50) NOT NULL  
);
```

En este ejemplo, se crea una tabla llamada "Ejemplo" con dos columnas: "Id" y "Nombre". La columna "Id" está definida como un uniqueidentifier con un valor predeterminado generado por la función "newid". La columna "Id" también se define como la clave primaria de la tabla.

En conclusión, la propiedad "identity" y la función "newid" son herramientas útiles en SQL Server para generar valores únicos en las tablas de la base de datos. La propiedad "identity" se utiliza para generar valores únicos y secuenciales para una

columna determinada en una tabla, mientras que la función "newid" se utiliza para generar valores únicos de tipo uniqueidentifier en una tabla. Ambas herramientas son útiles para garantizar la integridad de los datos en la base de datos.

Tipos de integridad

Integridad de dominio

La integridad de dominio es uno de los tipos de integridad de datos que se pueden aplicar en una base de datos. Consiste en definir restricciones en los valores que se pueden almacenar en un campo específico de una tabla, limitando así los valores posibles para ese campo. Estas restricciones se basan en un dominio, que es un conjunto de valores permitidos para un campo determinado.

Por ejemplo, en una tabla de empleados, se puede definir un campo "estado civil" que solo permita los valores "soltero", "casado" o "divorciado". Cualquier otro valor que se intente insertar en ese campo será rechazado por la base de datos debido a la restricción de integridad de dominio.

La integridad de dominio se puede aplicar utilizando la sentencia CHECK en la definición de la tabla. Por ejemplo, la tabla de empleados podría definirse de la siguiente manera:

```
CREATE TABLE empleados (  
  id INT PRIMARY KEY,  
  nombre VARCHAR(50),  
  estado_civil VARCHAR(10) CHECK (estado_civil IN ('soltero', 'casado', 'divorciado'))  
);
```

En este ejemplo, la restricción de integridad de dominio se establece mediante la cláusula CHECK en la definición de la columna "estado_civil", que solo permite los valores "soltero", "casado" o "divorciado".

La integridad de dominio es importante para garantizar la calidad y coherencia de los datos en una base de datos. Al limitar los valores permitidos para un campo determinado, se evita la entrada de datos incorrectos o inconsistentes, lo que puede provocar errores en las aplicaciones que utilizan la base de datos o incluso la corrupción de los datos.

Integridad de entidad

La integridad de entidad es un tipo de integridad de datos que asegura que cada fila de una tabla tenga una identidad única y que no haya filas duplicadas. En otras palabras, cada registro debe tener una clave única y no se permiten registros duplicados en la tabla.

La integridad de entidad se puede aplicar a través de la definición de una clave primaria en la tabla, que es un campo o una combinación de campos que identifican de forma única cada fila. La clave primaria se utiliza para garantizar que no se puedan insertar filas duplicadas en la tabla. Si se intenta insertar una fila con una clave primaria que ya existe, la base de datos rechazará la operación.

Por ejemplo, supongamos que tenemos una tabla de empleados y queremos asegurarnos de que cada empleado tenga una identidad única en la tabla. Podríamos definir la columna "id" como la clave primaria de la tabla:

```
CREATE TABLE empleados (  
id INT PRIMARY KEY,
```

```
nombre VARCHAR(50),  
salario DECIMAL(10,2),  
departamento VARCHAR(50)  
);
```

En este ejemplo, la columna "id" se define como la clave primaria de la tabla "empleados". Esto significa que cada empleado tendrá una identidad única en la tabla y no se permitirán filas duplicadas con el mismo valor de "id".

La integridad de entidad es importante para garantizar la calidad y consistencia de los datos en una base de datos. Al garantizar que cada fila tenga una identidad única, se evita la entrada de datos duplicados o inconsistentes, lo que puede provocar errores en las aplicaciones que utilizan la base de datos o incluso la corrupción de los datos.

Integridad referencial

La integridad referencial es un tipo de integridad de datos en la que se asegura que las relaciones entre tablas sean coherentes y precisas. Esto se logra mediante el uso de claves foráneas que establecen relaciones entre dos tablas.

En la integridad referencial, una tabla puede tener una o varias claves foráneas que hacen referencia a una columna o conjunto de columnas en otra tabla. La clave foránea es una restricción que garantiza que los valores en la columna que hace referencia a la tabla relacionada existan en la tabla relacionada. De esta manera, se garantiza que los datos relacionados en las tablas estén en sincronía y que no haya datos huérfanos sin referencias.

Por ejemplo, supongamos que tenemos dos tablas, "clientes" y "pedidos". La tabla "clientes" tiene una clave primaria "id_cliente" que identifica de manera única a cada cliente y la tabla "pedidos" tiene una columna "id_cliente" que hace referencia a la columna "id_cliente" en la tabla "clientes". Para garantizar la integridad referencial, se puede definir una restricción de clave foránea en la columna "id_cliente" de la tabla "pedidos", como sigue:

```
CREATE TABLE clientes (  
  id_cliente INT PRIMARY KEY,  
  nombre VARCHAR(50),  
  direccion VARCHAR(50)  
);
```

```
CREATE TABLE pedidos (  
  id_pedido INT PRIMARY KEY,  
  id_cliente INT,  
  fecha_pedido DATE,  
  total DECIMAL(10,2),  
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)  
);
```

En este ejemplo, la restricción de clave foránea en la columna "id_cliente" de la tabla "pedidos" asegura que cada valor en la columna "id_cliente" exista en la tabla "clientes". Si se intenta insertar un registro en la tabla "pedidos" con un valor en la columna "id_cliente" que no existe en la tabla "clientes", la base de datos rechazará la operación.

La integridad referencial es importante para mantener la coherencia y la precisión de los datos en una base de datos. Al garantizar que las relaciones entre las tablas sean correctas, se evita la entrada de datos huérfanos o inconsistentes que pueden provocar errores en las aplicaciones que utilizan la base de datos.

Restricciones

En SQL Server, las restricciones son reglas que se aplican a las tablas para garantizar que los datos se inserten, actualicen o eliminen de acuerdo con ciertas condiciones. Estas restricciones pueden definirse cuando se crea la tabla o después de su creación.

Existen varios tipos de restricciones en SQL Server, que incluyen:

Default

En SQL Server, un valor predeterminado (default) se utiliza para proporcionar un valor predeterminado a una columna de una tabla cuando no se especifica un valor explícitamente en la instrucción INSERT.

Cuando se crea una tabla en SQL Server, se puede definir un valor predeterminado para cada columna de la tabla utilizando la cláusula DEFAULT. El valor predeterminado puede ser una expresión constante, una función incorporada, una función definida por el usuario o un valor NULL.

Por ejemplo, para crear una tabla con un valor predeterminado para una columna:

```
CREATE TABLE Ejemplo  
(  
  ID INT PRIMARY KEY,  
  Nombre VARCHAR(50),  
  Edad INT DEFAULT 18  
);
```

En este ejemplo, la columna "Edad" tiene un valor predeterminado de 18. Si se inserta un registro en la tabla sin especificar un valor para la columna "Edad", se utilizará el valor predeterminado de 18.

También es posible alterar la definición de una columna para agregar o modificar un valor predeterminado:

```
ALTER TABLE Ejemplo  
ALTER COLUMN Edad INT DEFAULT 20;
```

En este caso, la columna "Edad" de la tabla "Ejemplo" se altera para tener un valor predeterminado de 20 en lugar de 18.

El uso de valores predeterminados puede ser muy útil para evitar errores en la inserción de datos y asegurar que las columnas tengan valores adecuados cuando se insertan nuevos registros en la tabla.

Check

En SQL Server, una restricción de CHECK se utiliza para limitar los valores que se pueden insertar o actualizar en una columna de una tabla. La restricción de CHECK se define como una expresión lógica que devuelve un valor verdadero o falso.

Cuando se crea una tabla, se puede agregar una restricción de CHECK a una columna utilizando la cláusula CHECK. Por ejemplo:

```
CREATE TABLE Ejemplo  
(  
  ID INT PRIMARY KEY,  
  Nombre VARCHAR(50),  
  Edad INT CHECK (Edad >= 18)  
);
```

En este ejemplo, la columna "Edad" está restringida por una restricción de CHECK para asegurarse de que solo se puedan insertar o actualizar valores que sean iguales o mayores a 18. Si se intenta insertar un valor de "Edad" menor a 18, se producirá un error de restricción.

También es posible agregar una restricción de CHECK a una columna existente utilizando la cláusula ALTER TABLE. Por ejemplo:

```
ALTER TABLE Ejemplo  
ADD CONSTRAINT CK_Edad CHECK (Edad >= 18);
```

En este caso, se agrega una restricción de CHECK llamada "CK_Edad" a la columna "Edad" de la tabla "Ejemplo". La restricción asegura que solo se puedan insertar o actualizar valores que sean iguales o mayores a 18.

El uso de restricciones de CHECK es útil para asegurar que los valores ingresados en una tabla cumplan con ciertas condiciones lógicas y limitar los errores de inserción de datos.

Primary Key

En SQL Server, una clave primaria o Primary Key (PK) es una restricción de integridad que garantiza la unicidad y la no nulidad de una o varias columnas de una tabla. La clave primaria se utiliza para identificar de forma exclusiva cada registro en una tabla. Al definir una clave primaria, SQL Server crea automáticamente un índice único para la columna o conjunto de columnas especificado como clave primaria. Esto mejora el rendimiento al buscar o actualizar datos de la tabla.

Para agregar una clave primaria a una tabla en SQL Server, se puede utilizar la cláusula PRIMARY KEY en la definición de la tabla o utilizando la cláusula ALTER TABLE después de la creación de la tabla. Por ejemplo:

-- Definir una clave primaria en la creación de la tabla

```
CREATE TABLE Ejemplo (  
    ID INT PRIMARY KEY,  
    Nombre VARCHAR(50)  
);
```

-- Agregar una clave primaria utilizando ALTER TABLE

ALTER TABLE Ejemplo

ADD CONSTRAINT PK_Ejemplo PRIMARY KEY (ID);

En este ejemplo, se crea una tabla llamada "Ejemplo" con una columna "ID" como clave primaria. En la segunda consulta, se agrega una restricción de clave primaria llamada "PK_Ejemplo" a la columna "ID" de la tabla "Ejemplo".

Solo se puede definir una clave primaria por tabla y no puede haber valores duplicados en la columna o conjunto de columnas especificado como clave primaria. Si se intenta insertar un valor duplicado en la clave primaria, se producirá un error de restricción.

Unique

En SQL Server, la restricción UNIQUE se utiliza para garantizar que los valores en una o varias columnas de una tabla sean únicos. Al igual que una clave primaria, una restricción UNIQUE impide la inserción de registros duplicados en una tabla.

La principal diferencia entre una clave primaria y una restricción UNIQUE es que una tabla solo puede tener una clave primaria, mientras que puede haber varias restricciones UNIQUE en una tabla. Además, a diferencia de una clave primaria, una restricción UNIQUE puede permitir valores nulos.

Para agregar una restricción UNIQUE a una columna o conjunto de columnas en SQL Server, se puede utilizar la cláusula UNIQUE en la definición de la tabla o utilizando la cláusula ALTER TABLE después de la creación de la tabla. Por ejemplo:

-- Definir una restricción UNIQUE en la creación de la tabla

```
CREATE TABLE Ejemplo (  
    ID INT,  
    Nombre VARCHAR(50),  
    CONSTRAINT UK_Ejemplo UNIQUE (Nombre)  
);  
  
-- Agregar una restricción UNIQUE utilizando ALTER TABLE  
ALTER TABLE Ejemplo  
ADD CONSTRAINT UK_Ejemplo2 UNIQUE (ID, Nombre);
```

En este ejemplo, se crea una tabla llamada "Ejemplo" con una restricción UNIQUE en la columna "Nombre". En la segunda consulta, se agrega otra restricción UNIQUE llamada "UK_Ejemplo2" a las columnas "ID" y "Nombre" de la tabla "Ejemplo".

Si se intenta insertar un valor duplicado en una columna con una restricción UNIQUE, se producirá un error de restricción.

Foreign Key

En SQL Server, una clave foránea (foreign key) se utiliza para crear una relación entre dos tablas. La clave foránea se compone de una o varias columnas de una tabla, las cuales hacen referencia a la clave primaria de otra tabla. La clave foránea garantiza que los valores de la columna de referencia siempre existan en la tabla principal.

Por ejemplo, si tenemos dos tablas, "Estudiantes" y "Cursos", podemos crear una relación entre ellas mediante el uso de una clave foránea en la tabla "Estudiantes". La clave foránea se definiría en la tabla "Estudiantes" haciendo referencia a la clave

primaria de la tabla "Cursos", que podría ser el código del curso. De esta forma, cuando se inserta un nuevo registro en la tabla "Estudiantes", la clave foránea garantiza que el código del curso ingresado ya exista en la tabla "Cursos".

Para agregar una clave foránea a una tabla en SQL Server, se puede utilizar la cláusula FOREIGN KEY en la definición de la tabla o mediante la cláusula ALTER TABLE después de la creación de la tabla. Por ejemplo:

-- Crear las tablas Estudiantes y Cursos

```
CREATE TABLE Cursos (  
    CodigoCurso INT PRIMARY KEY,  
    NombreCurso VARCHAR(50)  
);
```

```
CREATE TABLE Estudiantes (  
    ID INT PRIMARY KEY,  
    Nombre VARCHAR(50),  
    CodigoCurso INT,  
    CONSTRAINT FK_Estudiantes_Cursos FOREIGN KEY (CodigoCurso) REFERENCES  
Cursos(CodigoCurso)  
);
```

En este ejemplo, se crea la tabla "Cursos" con una clave primaria en la columna "CodigoCurso". Luego, se crea la tabla "Estudiantes" con una clave primaria en la columna "ID" y una clave foránea en la columna "CodigoCurso", que hace referencia a la clave primaria "CodigoCurso" de la tabla "Cursos". Si se intenta insertar un valor en la columna "CodigoCurso" de la tabla "Estudiantes" que no exista en la tabla "Cursos", se producirá un error de restricción.

Integridad referencial en cascada

La integridad referencial en cascada es una característica de las restricciones de clave foránea en SQL Server que permite automatizar la propagación de los cambios en las tablas relacionadas.

Cuando se establece una restricción de clave foránea con la opción "ON DELETE CASCADE" o "ON UPDATE CASCADE", esto significa que cualquier cambio en el valor de la clave primaria de la tabla referenciada se propagará automáticamente a la tabla que tiene la clave foránea correspondiente. Por ejemplo, si se elimina un registro de la tabla primaria, todos los registros de la tabla secundaria que referencian el registro eliminado también serán eliminados.

Es importante tener en cuenta que la integridad referencial en cascada debe utilizarse con precaución, ya que puede tener consecuencias no deseadas si se aplican incorrectamente. Por ejemplo, si se elimina accidentalmente un registro importante de la tabla primaria, se pueden eliminar muchos registros de la tabla secundaria si se ha configurado la restricción de cascada.

A continuación, se muestra un ejemplo de cómo crear una restricción de clave foránea en cascada en SQL Server:

```
CREATE TABLE Customers (  
    CustomerID int PRIMARY KEY,  
    CustomerName varchar(255),  
    ContactName varchar(255),  
    Country varchar(255)  
);
```

```
CREATE TABLE Orders (  
    OrderID int PRIMARY KEY,  
    CustomerID int,  
    OrderDate date,  
    FOREIGN KEY (CustomerID)  
        REFERENCES Customers(CustomerID)  
    ON DELETE CASCADE  
);
```

En este ejemplo, la tabla "Orders" tiene una clave foránea que hace referencia a la columna "CustomerID" de la tabla "Customers". La opción "ON DELETE CASCADE" está habilitada, lo que significa que cualquier cambio en la clave primaria de la tabla "Customers" se propagará automáticamente a la tabla "Orders".

Bibliografía y Webgrafía

- López, R. (2017). Introducción a las Bases de Datos. Desde Cero hasta Diseño de Sistemas. CreateSpace Independent Publishing Platform.
- Mistry, R. & Misner, S. (2013). SQL Server 2012. Guía del Administrador. Microsoft Press.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2011). Fundamentos de Bases de Datos. McGraw-Hill Interamericana.
- Mistry, R., & Varga, S. (2017). SQL Server 2016. Guía del Administrador. Microsoft Press.

Alter: comando utilizado para modificar la estructura de un objeto existente en la base de datos.

Check: restricción que permite especificar una condición que debe cumplir un valor ingresado en una columna.

Columna: estructura de una tabla que define un tipo de datos específico para almacenar información.

Creación y Eliminación: proceso de crear o eliminar objetos de la base de datos.

Create: comando utilizado para crear objetos de la base de datos, como tablas, vistas, procedimientos almacenados, entre otros.

Default: valor predeterminado que se asigna a una columna si no se especifica ningún valor al momento de ingresar datos.

Drop: comando utilizado para eliminar objetos de la base de datos.

Foreign Key: columna o conjunto de columnas que establecen una relación entre dos tablas.

Identificadores: nombres asignados a objetos de la base de datos, como tablas, columnas, procedimientos almacenados, etc.

Integridad de dominio: reglas que se aplican a los valores que se ingresan en las columnas de una tabla.

Integridad de entidad: reglas que se aplican a las tablas para asegurar que cada fila tenga un identificador único.

Integridad referencial en cascada: opción que permite realizar cambios en la tabla principal y que se reflejen automáticamente en las tablas secundarias que tienen una relación con ella.

Integridad referencial: reglas que aseguran que las relaciones entre tablas sean válidas y consistentes.

Lenguaje de Definición de Objetos: conjunto de comandos que permiten definir y manipular objetos de una base de datos.

Primary Key: columna o conjunto de columnas que identifican de manera única cada fila en una tabla.

Restricciones: reglas que se aplican a las columnas de una tabla para restringir los valores que pueden ser ingresados.

Tabla: objeto de la base de datos que almacena datos organizados en filas y columnas.

Tipos de datos definidos por el usuario: tipos de datos personalizados creados por el usuario, como enum o struct.

Tipos de Datos: clasificación de los datos que se almacenan en la base de datos, como enteros, decimales, cadenas de caracteres, fechas, entre otros.

Tipos de integridad: conjunto de reglas que aseguran que los datos almacenados en la base de datos son correctos y consistentes.

Unique: restricción que asegura que los valores en una columna o conjunto de columnas son únicos.

Uso de propiedad: identity, función newid: identificadores únicos asignados automáticamente a cada fila de una tabla, que pueden ser generados mediante la propiedad identity o la función newid.